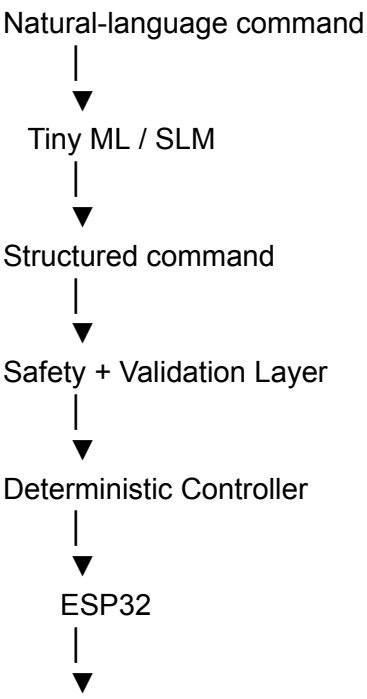
ESP32 TinyML Deployment Experiment — Complete Summary

1. What was the goal?

The ultimate goal of the project is to build a **resource-constrained intelligent washing-machine controller** in which an ML/SLM model interprets natural-language commands and produces a structured machine action.

The intended architecture is:



The challenge is that an ESP32 has **very limited RAM, Flash and compute compared with a desktop/GPU system**.

So instead of assuming that a model would run on the ESP32, we first needed experimental evidence that:

> **The ESP32 can execute a quantized neural network using an embedded inference runtime.**

That led to this staged experiment.

---

2. Why didn't we start directly with SmolLM2-135M?

Because that would mix together several independent problems:

- model size
- tokenizer requirements
- operator support
- RAM requirements
- inference runtime
- quantization
- Flash limitations
- computational cost
- task accuracy

If deployment failed, we wouldn't know **why** it failed.

Therefore we adopted a controlled progression:

Tiny model
↓
INT8 quantization
↓
TFLite conversion
↓
ESP32 integration
↓
Physical inference
↓
Latency
↓
RAM
↓
Flash
↓
Real task-specific model

The tiny model therefore acts as a **deployment compatibility probe**, not as the final washing-machine model.

---

3. Why did we choose TFLite + TFLite Micro?

This was one of the most important architectural decisions.

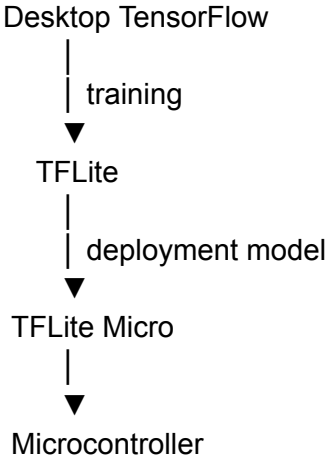
TensorFlow Lite / TFLite

TFLite provides a deployment-oriented representation of neural networks and supports **quantized models**, particularly INT8 models that are much more suitable for microcontrollers than full FP32 models.

TensorFlow Lite Micro

The ESP32 cannot simply run desktop TensorFlow.

TFLite Micro is designed specifically for microcontroller-class systems:



It uses a constrained memory model and allows us to provide a preallocated **tensor arena** rather than depending on a large dynamic memory environment.

For our ESP-IDF project we used Espressif's:

espressif/esp-tflite-micro

and its associated:

esp-nn

component.

This was preferable to attempting to run a full ML framework on the ESP32.

---

4. Why INT8?

This is critical for TinyML.

Our original model was converted from floating-point computation to **INT8 quantized computation**.

Instead of representing activations/weights as:

FP32 → 4 bytes/value

INT8 uses:

INT8 → 1 byte/value

with scale and zero-point parameters used to map between real values and quantized values.

Our ESP32 confirmed:

Input:
type = 9
scale = 0.012320
zero point = 127

Output:
type = 9
scale = 0.002214
zero point = 127

`type=9` corresponds to **INT8**.

Therefore this was not merely a TFLite model sitting on the ESP32—the hardware actually executed the **INT8 representation**.

---

5. The experimental model

We deliberately selected a very small neural network:

Input (1)
  │
  ▼
Dense(16) + ReLU
  │
  ▼
Dense(16) + ReLU
  │
  ▼
Dense(1)
  │

▼
Output

It was trained in Google Colab on a simple sine-function dataset.

The purpose wasn't to achieve impressive ML accuracy.

The purpose was:

> **Create a small, known-good neural network that can be converted to INT8 and used as a controlled hardware payload.**

This isolates the deployment problem.

---

6. Model conversion

The resulting models were:

Model	Size
FP32 TFLite	**3,164 B**
INT8 TFLite	**3,344 B**

Interestingly, the INT8 model was slightly larger.

That does **not** mean INT8 failed.

For such an extremely tiny network, quantization metadata can outweigh the savings from reducing a handful of parameters. This is exactly why model size should be **measured**, rather than assuming "INT8 = smaller file."

---

7. ESP32 integration

Hardware:

DOIT ESP32 DEVKIT V1
ESP32-D0WD-V3
4 MB Flash

We created an ESP-IDF project and used:

ESP-IDF v5.5.5

The TFLite Micro dependency was added through the ESP-IDF component manager:

espressif/esp-tflite-micro 1.4.0
espressif/esp-nn 1.3.2

The `.tflite` model was converted into a C++ byte array:

tiny_sine_int8_model.cc

and linked directly into the firmware.

So the deployment became:

tiny_sine_int8.tflite
↓
C++ byte array
↓
ESP-IDF firmware
↓
ESP32 Flash

This means the model does **not need a filesystem or external storage** to execute.

---

8. First milestone — model initialization

The first test only checked whether TFLite Micro could load the model.

The ESP32 reported:

Model loaded successfully.
Model schema version: 3
AllocateTensors() succeeded.
TFLite Micro model is running on ESP32!

Finding

PASS.

This established:

- model binary was valid
- TFLite schema was compatible
- TFLite Micro was correctly integrated
- required operators were available
- tensor memory could be allocated

This was our first proof of physical deployment.

---

9. Second milestone — actual inference

We then obtained the model's input/output tensors and performed:

interpreter.Invoke()

on the physical ESP32.

For:

x = 0

the value was quantized:

x = 0
↓
quantized = 127

and the model returned:

output = 0.000000

Since the reference function at zero is:

sin(0) = 0

the compatibility test behaved correctly for this input.

Finding

Actual neural-network inference on physical ESP32: PASS.

This is substantially stronger evidence than merely proving that the `.tflite` file exists in Flash.

---

10. Third milestone — inference latency

We then introduced a high-resolution ESP timer:

esp_timer_get_time()

and measured only the:

interpreter.Invoke()

operation.

We first performed a warm-up inference and then measured **100 inference runs**.

Results:

Metric	Result
Runs	**100**
Minimum latency	**88 µs**
Average latency	**89.20 µs**
Maximum latency	**129 µs**
Average	**0.0892 ms**

Therefore:

> **Average neural-network inference latency was 89.2 µs on the physical ESP32.**

This demonstrates that the tiny INT8 model is computationally inexpensive on the target MCU.

The commonly derived `~11,200 inferences/s` figure should be treated only as a theoretical reciprocal of the measured inference time—not as an end-to-end application throughput claim.

---

11. Fourth milestone — RAM

We then measured heap availability at multiple stages.

The ESP32 reported:

Before TFLite:
282,732 bytes

After model load:
282,732 bytes

After AllocateTensors():
282,548 bytes

After inference:
282,320 bytes

Therefore:

Dynamic heap change

Model loading:

0 B

Tensor allocation:

282732 - 282548
= 184 B

After inference:

282732 - 282320
= 412 B

However, there is an extremely important interpretation:

We allocated a 20 KB tensor arena

20,480 bytes

using:

static uint8_t tensor_arena[20 * 1024];

That memory is **statically allocated**, so it does not show up as a 20 KB drop in the dynamic heap measurement.

Therefore we **must not say the model consumes only 412 B RAM**.

The defensible statement is:

> The deployment reserved a 20 KB tensor arena, while the measured dynamic heap reduction was 184 B after tensor allocation and 412 B after inference.

---

12. Fifth milestone — Flash footprint

The final `idf.py size` analysis gave:

Memory region	Used
Flash Code	**87,410 B**
Flash Data	**52,300 B**
Total image	**202,921 B**
IRAM	**52,919 B**
DRAM	**33,828 B**
RTC SLOW	**56 B**

The application binary was approximately:

203 KB

and the 1 MB application partition had:

81% free

The tiny model itself remains only:

3,344 B ≈ 3.27 KiB

The ~203 KB firmware image includes much more than the model:

ESP-IDF
+ TFLite Micro
+ ESP-NN
+ application
+ model
+ runtime/support code

So again, we should **not claim that TFLite itself consumes 203 KB**.

---

13. Final experimental results

Our current hardware baseline is:

Category	Finding
Target	DOIT ESP32 DEVKIT V1
MCU	ESP32-D0WD-V3

Flash	4 MB
Framework	TFLite Micro
ESP-IDF	5.5.5
Model	Tiny Dense neural network
Quantization	**INT8**
TFLite model	**3,344 B**
Firmware image	**202,921 B**
App partition free	**81%**
Tensor arena	**20 KB reserved**
Dynamic heap Δ after allocation	**184 B**
Dynamic heap Δ after inference	**412 B**
Inference runs	**100**
Min latency	**88 µs**
Mean latency	**89.2 µs**
Max latency	**129 µs**
Physical inference	**PASS**

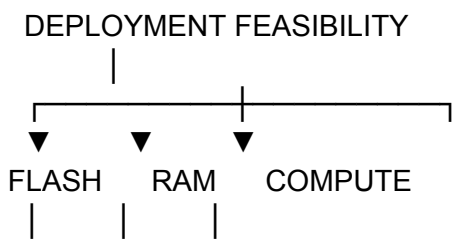
---

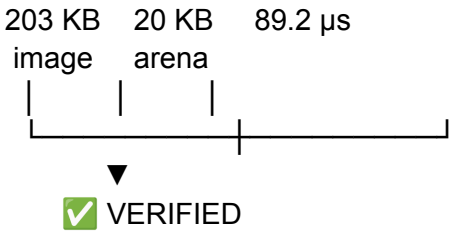
14. What have we actually proven?

The strongest conclusion is:

> **A quantized neural-network model can be successfully integrated into an ESP-IDF application, embedded in ESP32 Flash, initialized using TFLite Micro, allocated within a statically provisioned tensor arena, and executed as INT8 inference on the physical ESP32.**

And we have quantitative measurements for:





---

15. Why this path is important for the actual SLM project

This experiment gives us a **measured hardware baseline** before we introduce the real washing-machine intelligence.

Now we know the deployment stack works:

Training
↓
Quantization
↓
TFLite
↓
TFLite Micro
↓
ESP-IDF
↓
ESP32
↓
Physical inference

Therefore, when we eventually bring in the actual task model, we can ask concrete questions:

Can it fit?

Model size
+
Runtime
+
Tensor arena
<
ESP32 resources?

Is it fast enough?

Inference latency
<

application latency budget?

Is it accurate enough?

ESP32 model accuracy
vs
reference model accuracy

Is the memory footprint acceptable?

Flash + RAM + tensor arena

---

16. And this leads directly to the real model

Your existing project has a much more meaningful benchmark:

SmolLM2-135M-Instruct
+
repaired-label LoRA
↓
Structured washing-machine commands

with the established frozen benchmark:

41 / 45 exact = 91.11%

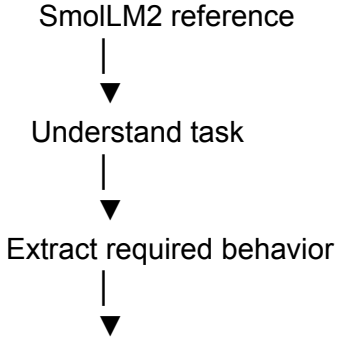
That should remain our **reference benchmark**.

But we now have to recognize that **SmolLM2-135M is a language model**, whereas our current ESP32 experiment is a tiny feed-forward neural network.

So the next phase isn't simply:

"Put SmolLM2 onto ESP32."

Instead, the engineering question becomes:



Buil**ESP32 TinyML Deployment Experiment — Complete Summary**

1. What was the goal?

The ultimate goal of the project is to build a **resource-constrained intelligent washing-machine controller** in which an ML/SLM model interprets natural-language commands and produces a structured machine action.

The intended architecture is:

Natural-language command
↓
Tiny ML / SLM
↓
Structured command
↓
Safety + Validation Layer
↓
Deterministic Controller
↓
ESP32
↓
Washing-machine hardware

The challenge is that an ESP32 has **very limited RAM, Flash and compute compared with a desktop/GPU system**.

So instead of assuming that a model would run on the ESP32, we first needed experimental evidence that:

> **The ESP32 can execute a quantized neural network using an embedded inference runtime.**

That led to this staged experiment.

---

2. Why didn't we start directly with SmolLM2-135M?

Because that would mix together several independent problems:

- model size
- tokenizer requirements
- operator support
- RAM requirements
- inference runtime
- quantization
- Flash limitations
- computational cost
- task accuracy

If deployment failed, we wouldn't know **why** it failed.

Therefore we adopted a controlled progression:

Tiny model
↓
INT8 quantization
↓
TFLite conversion
↓
ESP32 integration
↓
Physical inference
↓
Latency
↓
RAM
↓
Flash
↓
Real task-specific model

The tiny model therefore acts as a **deployment compatibility probe**, not as the final washing-machine model.

---

3. Why did we choose TFLite + TFLite Micro?

This was one of the most important architectural decisions.

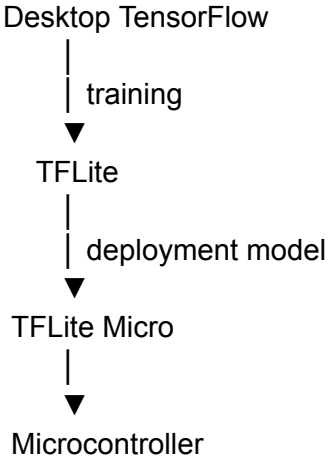
TensorFlow Lite / TFLite

TFLite provides a deployment-oriented representation of neural networks and supports **quantized models**, particularly INT8 models that are much more suitable for microcontrollers than full FP32 models.

TensorFlow Lite Micro

The ESP32 cannot simply run desktop TensorFlow.

TFLite Micro is designed specifically for microcontroller-class systems:



It uses a constrained memory model and allows us to provide a preallocated **tensor arena** rather than depending on a large dynamic memory environment.

For our ESP-IDF project we used Espressif's:

espressif/esp-tflite-micro

and its associated:

esp-nn

component.

This was preferable to attempting to run a full ML framework on the ESP32.

---

4. Why INT8?

This is critical for TinyML.

Our original model was converted from floating-point computation to **INT8 quantized computation**.

Instead of representing activations/weights as:

FP32 → 4 bytes/value

INT8 uses:

INT8 → 1 byte/value

with scale and zero-point parameters used to map between real values and quantized values.

Our ESP32 confirmed:

Input:
type = 9
scale = 0.012320
zero point = 127

Output:
type = 9
scale = 0.002214
zero point = 127

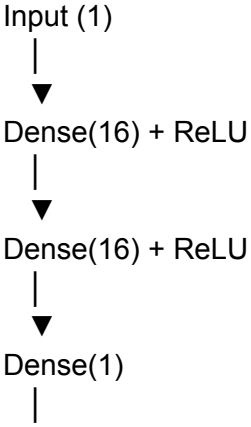
`type=9` corresponds to **INT8**.

Therefore this was not merely a TFLite model sitting on the ESP32—the hardware actually executed the **INT8 representation**.

---

5. The experimental model

We deliberately selected a very small neural network:



▼
Output

It was trained in Google Colab on a simple sine-function dataset.

The purpose wasn't to achieve impressive ML accuracy.

The purpose was:

> **Create a small, known-good neural network that can be converted to INT8 and used as a controlled hardware payload.**

This isolates the deployment problem.

---

6. Model conversion

The resulting models were:

Model	Size
FP32 TFLite	**3,164 B**
INT8 TFLite	**3,344 B**

Interestingly, the INT8 model was slightly larger.

That does **not** mean INT8 failed.

For such an extremely tiny network, quantization metadata can outweigh the savings from reducing a handful of parameters. This is exactly why model size should be **measured**, rather than assuming "INT8 = smaller file."

---

7. ESP32 integration

Hardware:

DOIT ESP32 DEVKIT V1
ESP32-D0WD-V3
4 MB Flash

We created an ESP-IDF project and used:

ESP-IDF v5.5.5

The TFLite Micro dependency was added through the ESP-IDF component manager:

espressif/esp-tflite-micro 1.4.0
espressif/esp-nn 1.3.2

The `.tflite` model was converted into a C++ byte array:

tiny_sine_int8_model.cc

and linked directly into the firmware.

So the deployment became:

tiny_sine_int8.tflite
↓
C++ byte array
↓
ESP-IDF firmware
↓
ESP32 Flash

This means the model does **not need a filesystem or external storage** to execute.

---

8. First milestone — model initialization

The first test only checked whether TFLite Micro could load the model.

The ESP32 reported:

Model loaded successfully.
Model schema version: 3
AllocateTensors() succeeded.
TFLite Micro model is running on ESP32!

Finding

PASS.

This established:

- model binary was valid
- TFLite schema was compatible
- TFLite Micro was correctly integrated
- required operators were available
- tensor memory could be allocated

This was our first proof of physical deployment.

---

9. Second milestone — actual inference

We then obtained the model's input/output tensors and performed:

interpreter.Invoke()

on the physical ESP32.

For:

x = 0

the value was quantized:

x = 0
↓
quantized = 127

and the model returned:

output = 0.000000

Since the reference function at zero is:

sin(0) = 0

the compatibility test behaved correctly for this input.

Finding

Actual neural-network inference on physical ESP32: PASS.

This is substantially stronger evidence than merely proving that the `.tflite` file exists in Flash.

---

10. Third milestone — inference latency

We then introduced a high-resolution ESP timer:

esp_timer_get_time()

and measured only the:

interpreter.Invoke()

operation.

We first performed a warm-up inference and then measured **100 inference runs**.

Results:

Metric	Result
Runs	**100**
Minimum latency	**88 µs**
Average latency	**89.20 µs**
Maximum latency	**129 µs**
Average	**0.0892 ms**

Therefore:

> **Average neural-network inference latency was 89.2 µs on the physical ESP32.**

This demonstrates that the tiny INT8 model is computationally inexpensive on the target MCU.

The commonly derived `~11,200 inferences/s` figure should be treated only as a theoretical reciprocal of the measured inference time—not as an end-to-end application throughput claim.

---

11. Fourth milestone — RAM

We then measured heap availability at multiple stages.

The ESP32 reported:

Before TFLite:
282,732 bytes

After model load:
282,732 bytes

After AllocateTensors():
282,548 bytes

After inference:
282,320 bytes

Therefore:

Dynamic heap change

Model loading:

0 B

Tensor allocation:

282732 - 282548
= 184 B

After inference:

282732 - 282320
= 412 B

However, there is an extremely important interpretation:

We allocated a 20 KB tensor arena

20,480 bytes

using:

static uint8_t tensor_arena[20 * 1024];

That memory is **statically allocated**, so it does not show up as a 20 KB drop in the dynamic heap measurement.

Therefore we **must not say the model consumes only 412 B RAM**.

The defensible statement is:

> The deployment reserved a 20 KB tensor arena, while the measured dynamic heap reduction was 184 B after tensor allocation and 412 B after inference.

---

12. Fifth milestone — Flash footprint

The final `idf.py size` analysis gave:

Memory region	Used
Flash Code	**87,410 B**
Flash Data	**52,300 B**
Total image	**202,921 B**
IRAM	**52,919 B**
DRAM	**33,828 B**
RTC SLOW	**56 B**

The application binary was approximately:

203 KB

and the 1 MB application partition had:

81% free

The tiny model itself remains only:

3,344 B ≈ 3.27 KiB

The ~203 KB firmware image includes much more than the model:

ESP-IDF
+ TFLite Micro
+ ESP-NN
+ application
+ model
+ runtime/support code

So again, we should **not claim that TFLite itself consumes 203 KB**.

---

13. Final experimental results

Our current hardware baseline is:

Category	Finding
Target	DOIT ESP32 DEVKIT V1
MCU	ESP32-D0WD-V3

Flash	4 MB
Framework	TFLite Micro
ESP-IDF	5.5.5
Model	Tiny Dense neural network
Quantization	**INT8**
TFLite model	**3,344 B**
Firmware image	**202,921 B**
App partition free	**81%**
Tensor arena	**20 KB reserved**
Dynamic heap Δ after allocation	**184 B**
Dynamic heap Δ after inference	**412 B**
Inference runs	**100**
Min latency	**88 µs**
Mean latency	**89.2 µs**
Max latency	**129 µs**
Physical inference	**PASS**

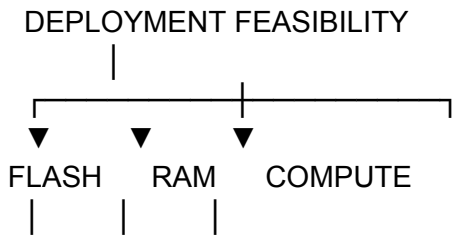
---

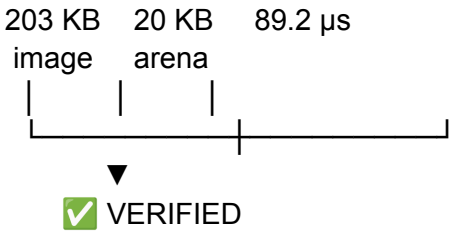
14. What have we actually proven?

The strongest conclusion is:

> **A quantized neural-network model can be successfully integrated into an ESP-IDF application, embedded in ESP32 Flash, initialized using TFLite Micro, allocated within a statically provisioned tensor arena, and executed as INT8 inference on the physical ESP32.**

And we have quantitative measurements for:





---

15. Why this path is important for the actual SLM project

This experiment gives us a **measured hardware baseline** before we introduce the real washing-machine intelligence.

Now we know the deployment stack works:

Training
↓
Quantization
↓
TFLite
↓
TFLite Micro
↓
ESP-IDF
↓
ESP32
↓
Physical inference

Therefore, when we eventually bring in the actual task model, we can ask concrete questions:

Can it fit?

Model size
+
Runtime
+
Tensor arena
<
ESP32 resources?

Is it fast enough?

Inference latency
<

application latency budget?

Is it accurate enough?

ESP32 model accuracy
vs
reference model accuracy

Is the memory footprint acceptable?

Flash + RAM + tensor arena

---

16. And this leads directly to the real model

Your existing project has a much more meaningful benchmark:

SmolLM2-135M-Instruct
+
repaired-label LoRA
↓
Structured washing-machine commands

with the established frozen benchmark:

41 / 45 exact = 91.11%

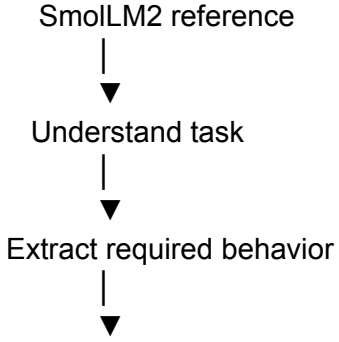
That should remain our **reference benchmark**.

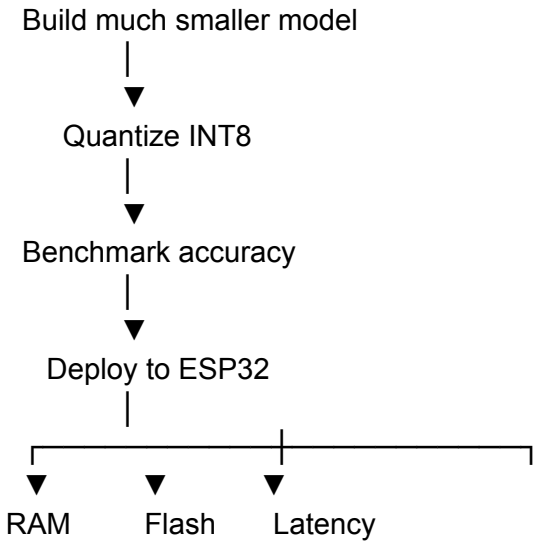
But we now have to recognize that **SmolLM2-135M is a language model**, whereas our current ESP32 experiment is a tiny feed-forward neural network.

So the next phase isn't simply:

"Put SmolLM2 onto ESP32."

Instead, the engineering question becomes:





That is the real TinyML research/development path.

We didn't waste time with the sine model—the sine model gave us the crucial evidence that the **entire embedded inference pipeline works on your actual ESP32**. Now we can move from *"can this hardware run a neural network?"* to the much more interesting question:

> **"What is the smallest model that can perform our washing-machine command-understanding task accurately enough while satisfying the ESP32's RAM, Flash and latency constraints?"**

That is where the experiment becomes a genuine **TinyML model-compression/deployment study**, rather than just an ESP32 demo

d much smaller model
Quantize INT8
Benchmark accuracy
Deploy to ESP32
RAM
Flash
Latency

That is the real TinyML research/development path.

We didn't waste time with the sine model—the sine model gave us the crucial evidence that the **entire embedded inference pipeline works on your actual ESP32**. Now we can move from *"can this hardware run a neural network?"* to the much more interesting question:

> **"What is the smallest model that can perform our washing-machine command-understanding task accurately enough while satisfying the ESP32's RAM, Flash and latency constraints?"**

That is where the experiment becomes a genuine **TinyML model-compression/deployment study**, rather than just an ESP32 demo

16-09-2026

SLM-PROJECT — Today's Findings & Progress

1. We clarified the actual objective

The goal is **not** merely to run a TinyML classifier on ESP32.

The real objective is:

> **Run a genuine autoregressive Small Language Model directly on the physical ESP32-D0WD-V3 and eventually use it for the washing-machine command-understanding system.**

The intended architecture remains:

Natural Language
↓
Tiny SLM
↓
Structured Command
↓
Safety + Validation Layer
↓
Deterministic Controller
↓
ESP32
↓
Washing Machine

Your existing **SmolLM2-135M-Instruct + LoRA** model remains the quality/reference benchmark. We're **not trying to squeeze that 135M model directly onto this original ESP32**.

Instead, we're first proving that a genuinely small transformer can execute on the hardware.

---

2. We established the TinyML deployment baseline

Before touching an actual language model, we deliberately used a tiny INT8 TFLite neural network to validate the embedded inference stack.

Hardware

Board : DOIT ESP32 DEVKIT V1
Chip : ESP32-D0WD-V3
Revision : v3.1
CPU : 160 MHz currently
Flash : 4 MB physical
PSRAM : None assumed
Port : COM6

Software

ESP-IDF : 5.5.5
TFLite Micro : espressif/esp-tflite-micro 1.4.0
ESP-NN : 1.3.2

The toy INT8 model successfully executed **real inference on the physical ESP32**.

Measured performance

100 Invoke() runs

Minimum : 88 µs
Average : 89.2 µs
Maximum : 129 µs

So we proved:

ESP-IDF
↓
Model embedding
↓
TFLite Micro
↓

Tensor allocation
↓
INT8 inference
↓
Physical ESP32

works.

That was our deployment smoke test.

---

3. We measured the ESP32's actual memory

This became one of today's most important findings.

On the clean baseline firmware:

Free heap : 304,052 B
Minimum free heap : 304,052 B
Largest free block : 172,032 B
Internal free RAM : 381,380 B

The distinction between total free RAM and **largest contiguous block** is important.

We cannot simply say:

> "ESP32 has ~381 KB, therefore any 381 KB allocation is possible."

The largest immediately usable contiguous block was only:

172 KB

This will strongly influence how we implement the transformer.

---

4. We investigated the actual TinyStories model

We chose **TinyStories 260K** as our first genuine SLM target.

It is a real autoregressive transformer—not a classifier pretending to be an LLM.

Model:

stories260K.bin

Size:

1,056,540 bytes

Configuration extracted from its checkpoint:

```
dim          = 64
hidden_dim   = 172
n_layers     = 5
n_heads      = 8
n_kv_heads   = 4
vocab_size   = 512
seq_len      = 512
```

We also have:

```
tok512.bin
size = 6,227 bytes
```

The exact model/tokenizer pairing was important.

---

5. We got the PC reference runtime working

The original `llama2.c` reference runtime initially crashed on Windows.

The important discovery was that:

stories260K.bin

requires its matching:

tok512.bin

rather than the repository's default tokenizer.

Once we used:

```
stories260K.bin
+
tok512.bin
```

the PC runtime successfully generated text.

Example:

> "Once upon a time, there was a little girl named Lily..."

So we now have a **golden reference** for what the ESP32 implementation should eventually reproduce.

PC reference:

```
TinyStories 260K
      ↓
llama2.c
      ↓
valid autoregressive generation
```

---

6. We analyzed the transformer architecture

The PC implementation showed the actual computation path:

```
Token
  ↓
Embedding
  ↓
5 Transformer Layers
  │
  ├── RMSNorm
  ├── Q/K/V projections
  ├── RoPE
  ├── Multi-head attention
  ├── Output projection
  ├── Residual
  │
  ├── RMSNorm
  ├── W1 + W3
  ├── SiLU
  ├── Elementwise multiply
  ├── W2
  └── Residual
  ↓
Final RMSNorm
  ↓
Classifier
```

↓
512 logits
↓
Next token

We also identified matrix multiplication as a major computational hotspot.

This is the actual SLM we are trying to port.

---

7. We identified the big RAM challenge

From the model configuration:

```
dim      = 64
n_heads  = 8
n_kv_heads = 4
```

we calculated:

```
kv_dim = 64 × 4 / 8
       = 32
```

With the original context length of 512 and float32 KV cache:

```
Key cache   ≈ 327,680 B
Value cache ≈ 327,680 B

Total       ≈ 655,360 B
            ≈ 640 KiB
```

That is far beyond the available RAM budget of the original ESP32.

So we already know:

> **We cannot blindly port the PC implementation unchanged.**

We'll need a memory-aware implementation—potentially involving reduced context, lower-precision storage, different KV handling, or another strategy.

But we're **not choosing that strategy prematurely**. We want to measure each stage on real hardware first.

---

8. We created a completely separate ESP32 LLM project

Project:

D:\HAVELLS-PROJ\SLM\esp32_llm_test\esp32_llm_runtime

This is separate from the earlier TFLite experiment.

Current structure:

```
esp32_llm_runtime/
│
├── CMakeLists.txt
│
└── main/
    ├── CMakeLists.txt
    ├── esp32_llm_runtime.cpp
    │
    └── model/
        ├── stories260K.bin
        ├── stories260K_model.cc
        └── stories260K_model.h
```

---

9. We successfully embedded the real model

We converted:

stories260K.bin

into a C++ byte array.

Original:

1,056,540 bytes

Generated source:

stories260K_model.cc
≈ 5.68 MB as text

The `.cc` file is much larger because:

0x40,0x00,0x00,...

represents every binary byte as textual source.

That **doesn't mean the model became 5.68 MB in firmware**.

The actual embedded binary remains approximately:

1.056 MB

---

10. We solved a C/C++ linkage issue

We initially hit:

undefined reference to `stories260K_model'
undefined reference to `stories260K_model_len'

The reason was the interaction between C++ `const` global linkage and our generated definitions.

We fixed the generated model definitions to:

```
extern const unsigned char stories260K_model[] = {
    ...
};

extern const unsigned int stories260K_model_len = 1056540;
```

After that, the linker succeeded.

This is significant because it means the actual checkpoint is now genuinely part of the ESP32 application.

---

11. The original 1 MB Flash partition was too small

After embedding the model, the firmware became:

1,220,288 bytes

But the original factory partition was:

1,048,576 bytes

So ESP-IDF correctly rejected it:

app partition is too small
overflow = 170,688 bytes

This wasn't a model/runtime failure.

It was simply:

Model + firmware
↓
1.22 MB
↓
1 MB partition ❌

---

12. We used the ESP32's available Flash properly

Your physical chip was detected as:

4 MB

but the firmware header had been configured for:

2 MB

and the application partition was only 1 MB.

We changed the partition table from:

Single factory app, no OTA

to:

Single factory app (large), no OTA

Result:

factory partition = 1500 KB

Now:

Firmware ≈ 1,220,288 B
Partition = 1,536,000 B
Free ≈ 316,736 B

---

13. The complete model now boots on the physical ESP32

This is probably the biggest milestone of today.

We flashed the new firmware and got:

ESP32 TinyStories checkpoint test

Checkpoint address : 0x3f40738c
Checkpoint size   : 1056540 bytes

Free heap   : 304052 bytes
Minimum free heap : 304052 bytes
Largest free block : 172032 bytes
Internal free RAM : 381380 bytes

Header bytes:
40 00 00 00 AC 00 00 00

The ESP32 then returned from:

app_main()

without crashing.

So we have physically demonstrated:

ESP32-D0WD-V3
|
▼
1.056 MB checkpoint
|
▼
Flash memory
|
▼
CPU can access it
|
▼
Header verified
|
▼
BOOT ✅

🏁 Today's final state

If I reduce the entire day to one engineering statement:

> **We have successfully taken a real 260K-parameter autoregressive transformer checkpoint from the PC reference environment, embedded the actual 1.056 MB checkpoint into an ESP-IDF application, allocated enough Flash for it, flashed it onto the physical ESP32-D0WD-V3, and verified that the ESP32 can directly access the checkpoint while retaining ~304 KB free heap and a ~172 KB largest contiguous block.**

That's a **very solid milestone**.

Current status

Stage	Status
ESP32 hardware verified	✅
ESP-IDF 5.5.5	✅
TinyML inference smoke test	✅
Physical RAM baseline	✅
TinyStories PC reference	✅
TinyStories checkpoint obtained	✅
Correct tokenizer identified	✅
ESP32 LLM project	✅
Checkpoint embedded	✅
Linker issue fixed	✅
Flash partition expanded	✅
Model flashed to ESP32	✅
Checkpoint accessible on ESP32	✅
Checkpoint header verified	✅
Transformer inference on ESP32	⏳

Tokenizer on ESP32 ⏳

Autoregressive generation ⏳

🚀 And now the fun part

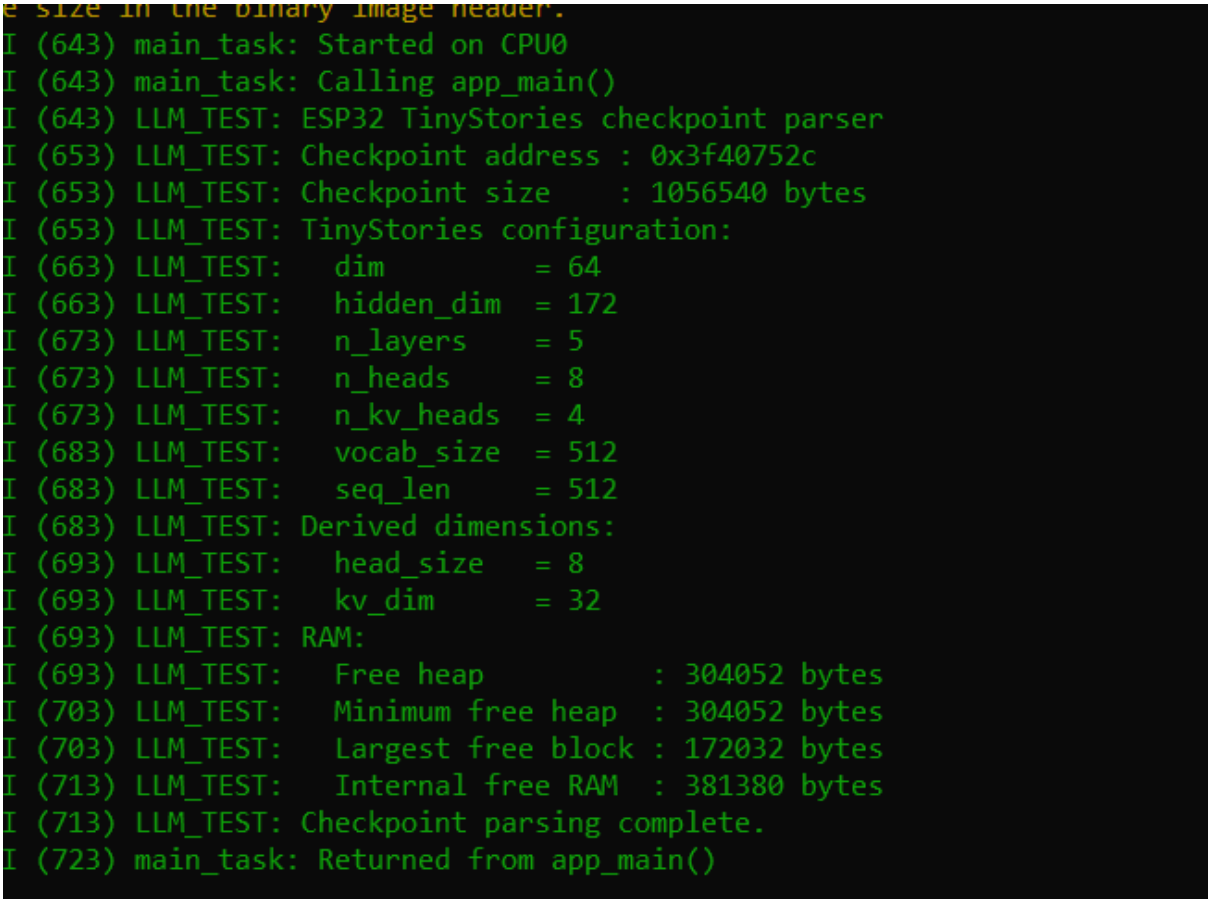
We're exactly at the boundary between:

"ESP32 contains an SLM"

and

"ESP32 actually IS running the SLM."

Next, we'll make the ESP32 **parse the checkpoint configuration directly from Flash** and verify all seven model parameters on-device. Then we can start implementing the actual transformer computation incrementally—**one token first, generation later.**



This is critical because we **do not want to copy the entire 1.056 MB checkpoint into RAM.**

The ideal architecture is:



That's how we have a chance of fitting this onto the original ESP32.